Produzindo Programas Mais Seguros

A demonstração de correção procura verificar que determinado programa ou segmento de programa para computadores está de acordo com suas especificações. Como vimos, essa abordagem depende da lógica formal para provar que determinada relação (a precondição) entre as variáveis do programa é válida antes da execução de determinado comando e que outra relação (a pós-condição) é válida depois da execução do comando. Como a natureza da demonstração de correção envolve um trabalho muito intenso, seu uso fica reservado, em geral, para seções críticas do código em aplicações importantes.

O método B é um conjunto de ferramentas que faz o seguinte:

1. Apoia a especificação formal de um projeto por meio de um modelo abstrato do sistema a ser desenvolvido. Esse apoio inclui tanto a geração automática de lemas que precisam ser provados para garantir que o modelo reflete os requisitos do sistema quanto ferramentas de demonstração automática para provar cada lema ou marcá-lo para verificação humana.
2. Traduz o modelo abstrato em um projeto com código pronto, novamente usando lemas para garantir que o projeto está de acordo com o modelo abstrato. O nível final então pode ser colocado em código, muitas vezes usando a linguagem de programação Ada, descrita por seus proponentes como "a linguagem projetada para construir sistemas realmente importantes".

Uma das aplicações mais interessantes da demonstração de correção baseada no método B é o desenvolvimento do programa para o trem Meteoro em Paris. Essa parte do sistema de trens de metrô de Paris foi projetada para transportar até 40.000 passageiros por hora e por sentido, com um intervalo entre trens tão curto como 85 segundos nas horas de pico. A parte crítica de segurança do programa inclui o andar e o parar de cada trem, o abrir e o fechar das portas, a energia para a tração elétrica, as rotas, a velocidade de cada trem e os alarmes de passageiros. No final do projeto, 27.800 lemas tinham sido demonstrados, com 92% deles de forma automática (sem intervenção humana). Mas eis a parte mais surpreendente: o número de *bugs* no código Ada encontrados pelos testes no computador hospedeiro, no computador alvo, no sítio e depois de o sistema ser colocado em operação foi ... 0. Zero, nada, nenhum. Impressionante!

Outros sistemas de métodos formais vêm sendo usados para projetos de programas críticos, como

- Desenvolvimento de um dispositivo de assistência para o ventrículo esquerdo que ajuda a bombear o sangue de pessoas com insuficiência cardíaca congestiva. O objetivo final é um coração artificial.
- "Algoritmos de detecção e resolução de conflitos" para a segurança no controle de tráfego aéreo.
- Desenvolvimento do programa Tokeneer ID Station para executar identificação biométrica de uma pessoa buscando acesso a um ambiente computacional seguro. Tokeneer é um sistema hipotético proposto pela NSA (National Security Agency, a Agência de Segurança Nacional dos EUA) como um problema desafiador para pesquisadores da área de segurança.

Formal Verification of Large Software Systems , Yin, X., and Knight, J., Proceedings of the NASA Formal Methods Symposium, April 13–15, 2010, Washington D.C., USA.

http://libre.adacore.com/academia/projects-single/echo

http://shemesh.larc.nasa.gov/fm/fm-atm-cdr.html

"Météor: A Successful Application of B in a Large Project," Behm, P., Benoit, P., Faivre, A., and Meynadier, J., *World Congress On Formal Methods in the Development of Computing Systems,* Toulouse, France, 1999, vol. 1709, pp. 369–387.

SEÇÃO 2.3 REVISÃO

TÉCNICAS

- Verificação da correção de um segmento de programa que inclui uma proposição com laço.
- Cálculo do mdc(a, b) usando o algoritmo de Euclides.

IDEIAS PRINCIPAIS

- Podemos usar um invariante do laço, demonstrando por indução no número de iterações, para provar a correção de um laço em um programa.
- Pode-se demonstrar que o algoritmo de Euclides clássico para encontrar o máximo divisor comum de dois números inteiros positivos está correto.

EXERCÍCIOS 2.3

1. Seja Q: $x^2 > x + 1$, em que x é um inteiro positivo. Suponha que Q é verdadeira depois da k-ésima iteração do laço de enquanto a seguir; prove que Q é verdadeira depois da próxima iteração.

   enquanto ($x > 5$) e ($x < 40$) **faça**
   $x = x + 1$
   fim do enquanto

2. Seja Q: $x! > 3^x$, em que x é um inteiro positivo. Suponha que Q é verdadeira depois da k-ésima iteração do laço de enquanto a seguir; prove que Q é verdadeira depois da próxima iteração.

   enquanto ($x > 10$) e ($x < 30$) **faça**

$x = x + 2$

fim do enquanto

Nos Exercícios de 3 a 6, prove que o pseudocódigo do segmento de programa está correto provando o invariante do laço Q e calculando Q depois de o laço terminar.

3. Função que retorna o valor de $x!$ para $x \geq 1$.

Fatorial (inteiro positivo x)
Variáveis locais:
inteiros i, j
$i = 2$
$j = 1$
enquanto $i \neq x + 1$ **faça**
$j = j * i$
$i = i + 1$
fim do enquanto
//agora j tem o valor $x!$
retorne j
fim da função *Fatorial*

Q: $j = (i - 1)!$

4. Função que retorna o valor de x^2 para $x \geq 1$

Quadrado (inteiro positivo x)
Variáveis locais:
inteiros i, j
$i = 1$
$j = 1$
enquanto $i \neq x$ **faça**
$j = j + 2i + 1$
$i = i + 1$
fim do enquanto
//agora j tem o valor x^2
retorne j
fim da função *Quadrado*

Q: $j = i^2$

5. Função que retorna o valor de x^y para $x, y \geq 1$

Potência (inteiro positivo x; inteiro positivo y)
Variáveis locais:
inteiros i, j
$i = 1$
$j = x$
enquanto $i \neq y$ **faça**
$j = j * x$
$i = i + 1$
fim do enquanto
//agora j tem o valor x^y
retorne j
fim da função *Potência*

Q: $j = x^i$

6. Função para calcular e escrever o quociente q e o resto r quando x é dividido por y, $x \geq 0$, $y \geq 1$

Divide (inteiro não negativo x; inteiro positivo y)
Variáveis locais:
inteiros não negativos q, r
$q = 0$
$r = x$
enquanto $r \geq y$ **faça**
$q = q + 1$
$r = r - y$
fim do enquanto
//q e r agora são o quociente e o resto
escreva("O quociente é" q "e o resto é" r)
fim da função *Divide*

Q: $x = q * y + r$

Para os Exercícios 7 a 12, use o algoritmo de Euclides para encontrar o máximo divisor comum dos números dados.

7. (308, 165) 8. (2420, 70)
9. (735, 90)
10. (8370, 465)
11. (1326, 252)
12. (1018215, 2695)
13. O problema a seguir foi apresentado no início do capítulo.

    > Você trabalha como voluntário em uma ONG que recebeu doações de 792 sabonetes e 400 frascos de xampus. Você quer fazer pacotes para distribuir em abrigos para pessoas sem teto de modo que todos os pacotes contenham o mesmo número de frascos de xampus e o mesmo número de sabonetes. Quantos pacotes você pode formar?

    Explique por que a solução deste problema é mdc(792, 400).
14. Em relação à pergunta apresentada no Exercício 13:
    a. Use o algoritmo de Euclides para encontrar o número de pacotes.
    b. Quantos frascos de xampus há em cada pacote?
    c. Quantos sabonetes há em cada pacote?

Nos Exercícios 15 a 21, prove que o segmento de programa está correto encontrando e demonstrando o invariante do laço apropriado Q e calculando Q depois de o laço terminar.

15. Função que retorna o valor de $x\,y^n$ para $n \geq 0$

    Cálculo (inteiro x; inteiro y; inteiro não negativo n)
    Variáveis locais:
    inteiros i, j
    $i = 0$
    $j = x$
    enquanto $i \neq n$ **faça**
    $j = j * y$
    $i = i + 1$
    fim do enquanto
    //j agora tem o valor de $x * y^n$
    retorne j
    fim da função *Cálculo*
16. Função que retorna o valor de $x\,y$ para $x, y \geq 0$

    Diferença (inteiro não negativo x; inteiro não negativo y) Variáveis locais:
    inteiros i, j
    $i = 0$
    $j = x$
    enquanto $i \neq y$ **faça**
    $j = j - 1$
    $i = i + 1$
    fim do enquanto
    //j agora tem o valor de $x - y$
    retorne j
    fim da função *Diferença*
17. Função que retorna o valor de $(x + 1)^2$ para $x \geq 1$

    QuadradoDoIncremento (inteiro positivo x)
    Variáveis locais:
    inteiros i, j
    $i = 1$
    $j = 4$
    enquanto $i \neq x$ **faça**
    $j = j + 2i + 3$
    $i = i + 1$
    fim do enquanto
    //j agora tem o valor de $(x + 1)^2$
    retorne j
    fim da função *QuadradoDoIncremento*
18. Função que retorna o valor de 2^n para $n \geq 1$

    PotênciasDe2 (inteiro positivo n)
    Variáveis locais:
    inteiros i, j
    $i = 1$
    $j = 2$
    enquanto $i \neq$ n **faça**
    $j = j * 2$

```
        i = i + 1
    fim do enquanto
    //j agora tem o valor de 2^n
    retorne j
fim da função PotênciaDe2
```

19. Função que retorna o valor de $x * n!$ para $n \geq 1$

```
Outra (inteiro x; inteiro positivo n) Variáveis locais:
inteiros i, j
    i = 1
    j = x
    enquanto i ≠ n faça
        j = j * (i + 1)
        i = i + 1
    fim do enquanto
    //j agora tem o valor de x * n!
    retorne j
fim da função Outra
```

20. Função que retorna o valor do polinômio $a_n x^n + a_{n-1}x^{n-1} + ... + a_1 x + a_0$ para um valor dado de x.

```
Poli (real a_n;...; real a_0; real x) Variáveis locais:
inteiros i, j
    i = n
    j = a_n
    enquanto i ≠ 0 faça
        j = j * x + a_{i-1}
        i = i – 1
    fim do enquanto
    //j agora tem o valor do polinômio em x
    retorne j
fim da função Poli
```

21. Função que retorna o valor máximo de $a[1], a[2], ..., a[n]$, $n \geq 1$, em um array de inteiros distintos.

```
MáxDoArray(inteiros n, a[1], a[2], ..., a[n])
Variáveis locais:
inteiros i, j
    i = 1
    j = a[1]
    enquanto i ≠ n faça
        i = i + 1
        se a[i] > j então j = a[i]
    fim do enquanto
    //j agora tem o valor do maior elemento do array
    retorne j
fim da função MáxDoArray
```

22. São dadas a seguir quatro funções que deveriam retornar o valor $a[1] + a[2] + ... + a[n]$ para $n \geq 1$ (a soma dos n primeiros elementos de um array de inteiros). Para as que não produzem resultados corretos, explique o que está errado. Para as que produzem resultados corretos, demonstre a correção.

a. *SomaA* (inteiros n, $a[1], a[2], ..., a[n]$)

```
Variáveis locais:
inteiros i, j
    i = 0
    j = 0
    enquanto i ≤ n faça
        j = j + a[i]
        i = i + 1
    fim do enquanto
    //j agora tem o valor a[1] + a[2] + ... + a[n]
    retorne j
fim da função SomaA
```

b. *SomaB* (inteiros n, $a[1], a[2], ..., a[n]$)

```
Variáveis locais:
inteiros i, j
    i = 1
    j = 0
    enquanto i ≤ n faça
        j = j + a[i]
```

```
        i = i + 1
      fim do enquanto
      //j agora tem o valor a[1] + a[2] + ... + a[n]
      retorne j
    fim da função SomaB
```

c. *SomaC* (inteiros *n*, *a*[1], *a*[2], ..., *a*[*n*])

```
    Variáveis locais:
    inteiros i, j
      i = 0
      j = 0
      enquanto i ≤ n faça
        j = j + a[i]
        i = i + 1
      fim do enquanto
      //j agora tem o valor a[1] + a[2] + ... + a[n]
      retorne j
    fim da função SomaC
```

d. *SomaD* (inteiros *n*, *a*[1], *a*[2], ..., *a*[*n*])

```
    Variáveis locais:
    inteiros i, j
      i = 1
      j = a[1]
      enquanto i ≤ n faça
        j = j + a[i + 1]
        i = i + 1
      fim do enquanto
      //j agora tem o valor a[1] + a[2] + ... + a[n]
      retorne j
    fim da função SomaD
```

Os Exercícios 23 a 28 tratam de uma variação do algoritmo de Euclides mais adequada à implementação computacional. O algoritmo MDC original depende de divisões inteiras repetidas para calcular restos. A variação a seguir, chamada de *algoritmo MDC binário*, também usa divisão, mas só por 2, mais subtração e um teste de paridade (para ver se o número é par ou ímpar). Como os números são armazenados no computador em forma binária, essas operações são simples e são efetuadas muitas vezes através de circuitos na placa. O teste par/ímpar pode ser feito usando-se conjunção bit a bit de N & 1, que resultará em 1 se e somente se N for ímpar. Dado um N ímpar, a divisão por 2 é obtida facilmente por um deslocamento de 1 bit à direita, em que todos os bits são deslocados uma casa para a direita (o bit na extrema direita desaparece) e o bit na extrema esquerda recebe o valor 0. (Multiplicação por 2 é um deslocamento de 1 bit para a esquerda.) A subtração envolve o complemento de 2. Como resultado, o algoritmo mdc binário, que evita a divisão regular, é mais rápido do que o algoritmo de Euclides, embora execute mais operações (mais simples). O algoritmo mdc binário baseiase em três fatos:

1. Se a e b forem ambos pares, então mdc(a, b) = 2mdc(a/2, b/2).
2. Se a for par e b for ímpar, então mdc(a, b) = mdc(a/2, b).
3. Se a e b forem ambos ímpares com $a \leq b$, então mdc(a, b) = mdc((a b)/2, b).

23. Para provar que, se a e b forem ambos pares, então mdc(a, b) = 2mdc(a/2, b/2), sejam a e b dois inteiros pares. Então 2 é um fator comum de a e de b, logo 2 é um fator de mdc(a, b). Seja $2c$ = mdc(a, b). Então

$$a = n(2c) \text{ e } b = m(2c)$$
$$a/2 = nc \text{ e } b/2 = mc$$

de modo que $c \mid a/2$ e $c \mid b/2$. Termine esta demonstração mostrando que c = mdc(a/2, b/2).

24. Para provar que, se a for par e b for ímpar, então mdc(a, b) = mdc(a/2, b), note que, como b é ímpar, 2 não é um fator de b, logo não é um fator de mdc(a, b). Logo, todas as contribuições para mdc(a, b) vêm de b e de a/2, portanto mdc(a, b) = mdc(a/2, b). Escreva uma equação para mdc(a, b) quando a for ímpar e b for par.

25. Queremos provar que, se a e b forem ambos ímpares com $a \geq b$, então mdc(a, b) = mdc((a b)/2, b). Se a e b forem ambos ímpares, então mdc(a, b) = mdc(a b, b) por causa do algoritmo de Euclides usual, pois o primeiro passo no cálculo de mdc(a, b) é $a = qb + r$, $0 \leq r < b$, enquanto o primeiro passo no cálculo de mdc($a - b$, b) é $a - b = (q - 1)\ b + r$, $0 \leq r < b$. O próximo passo nos dois casos é dividir b por r, de modo que as duas respostas finais terão que ser iguais. Termine esta demonstração mostrando que mdc($a - b$, b) = mdc(($a - b$)/2, b).

26. Para encontrar o mdc(420, 66), o algoritmo MDC binário executa os passos a seguir (compare com o Exemplo 27). Você pode fazer essas contas de cabeça!

420	66	Fator 1	Guarde o fator 2 para multiplicar ao final
210	33	Fator 2	
105	33	Fator 3	
36	33	Fator 2	

18	33	Fator 2	
9	33	Fator 3	[Como mdc(a, b) = mdc(b, a), não importa se o número maior é o primeiro ou o segundo, ou se o número par é o primeiro ou o segundo.]
9	12	Fator 2	
9	6	Fator 2	
9	3	Fator 3	
3	3	Fator 3	
0	3		

Um número agora é 0, logo o outro é um fator no mdc, logo mdc(420, 66) = 2 * 3 = 6 (o fator 2 vem do primeiro passo). Use o algoritmo MDC binário para calcular mdc(24, 20).

27. Use o algoritmo MDC binário para calcular mdc(308, 165) [veja o Exercício 7].
28. Use o algoritmo MDC binário para calcular mdc(2420, 70) [veja o Exercício 8].

SEÇÃO 2.4 TEORIA DOS NÚMEROS

Na Seção 2.1, demonstramos diversos resultados elementares de teoria dos números, como "o produto de dois inteiros pares é par". Essas demonstrações dependeram de definições básicas e das técnicas usuais de demonstração (demonstração direta, demonstração pela contrapositiva e demonstração por absurdo). Agora que temos munição adicional, podemos provar mais resultados sobre teoria dos números. A teoria dos números é divertida porque podem ser feitas conjecturas facilmente — afinal, os resultados são só para inteiros —, mas, apesar disso, as demonstrações podem ser muito difíceis. Um caso extremo é o **último teorema de Fermat**, que afirma que não existem inteiros positivos x, y e z para os quais

$$x^n + y^n = z^n$$

para qualquer inteiro $n > 2$. (Existem soluções para $n = 2$, como $3^2 + 4^2 = 5$.) Pierre de Fermat afirmou esse resultado em torno de 1637, mas — embora muitas "demonstrações" falsas tenham sido publicadas nesse ínterim — só em 1995 é que foi encontrada uma demonstração, quando o Dr. Andrew Wiles, da Universidade de Princeton, provou o resultado usando matemática bem complicada. Estamos interessados em teoria dos números, no entanto, por causa de sua utilidade em segurança computacional (veja a Seção 5.6).

O Teorema Fundamental da Aritmética

Vamos começar expandindo um resultado que provamos usando o segundo princípio de indução no Exemplo 23, Seção 2.2: para todo inteiro $n \geq 2$, n é um número primo ou é um produto de números primos.

De fato, pode ser feita uma afirmação mais forte.

LEMBRETE

Um número primo é um inteiro > 1 que só é divisível por ele mesmo e por 1.

TEOREMA TEOREMA O TEOREMA FUNDAMENTAL DA ARITMÉTICA

Para todo inteiro $n \geq 2$, n é um número primo ou pode ser escrito de maneira única (a menos de ordem) como um produto de números primos.

A parte nova aqui é que só existe um modo de fatorar um número composto como um produto de primos se ignorarmos a ordem dos fatores. Assim consideramos

$$2(3)(3) = 3(2)(3)$$

como a mesma fatoração de 18. Aceitamos, intuitivamente, a ideia da unicidade — de que outra maneira poderíamos escrever 18 como um produto de primos? Uma demonstração formal precisa de um pouco de preparação e começa com um novo olhar sobre o máximo divisor comum de dois inteiros positivos.

O algoritmo de Euclides para encontrar mdc(a, b) foi dado na Seção 2.3. Ocorre que, se a e b forem inteiros positivos, então mdc(a, b) sempre pode ser escrito como uma **combinação linear** de a e de b; ou seja,

$$\text{mdc}(a, b) = ia + jb \text{ para algum inteiro } i \text{ e algum inteiro } j$$

EXEMPLO 29

Usando o algoritmo de Euclides, vimos, na Seção 2.3, que mdc(420, 66) = 6. E 6 pode ser escrito como uma combinação linear de 420 e 66:

$$6 = 3(420) - 19(66)$$

Embora seja fácil verificar neste exemplo que 3(420) – 19(66) tem o valor 6, os valores dos coeficientes 3 e –19 parecem misteriosos. Eles não foram porém simplesmente tirados de uma cartola mágica; de fato, eles são obtidos das divisões sucessivas efetuadas pelo algoritmo de Euclides.

EXEMPLO 30

As divisões sucessivas executadas pelo algoritmo de Euclides para encontrar mdc(420, 66) podem ser escritas na seguinte forma (veja o Exemplo 27 na Seção 2.3):

$$\begin{aligned} 420 &= 6 \cdot 66 + 24 \\ 66 &= 2 \cdot 24 + 18 \\ 24 &= 1 \cdot 18 + 6 \\ 18 &= 3 \cdot 6 + 0 \end{aligned}$$

Reescrevendo as três primeiras equações de baixo para cima, obtemos

$$\begin{aligned} 6 &= 24 - 1 \cdot 18 \\ 18 &= 66 - 2 \cdot 24 \\ 24 &= 420 - 6 \cdot 66 \end{aligned}$$

Agora usamos essas equações fazendo diversas substituições:

$$\begin{aligned} 6 = 24 - 1 \cdot 18 &= 24 - 1 \cdot (66 - 2 \cdot 24) \text{ (substituindo por 18)} \\ &= 3 \cdot 24 - 66 \\ &= 3 \cdot (420 - 6 \cdot 66) - 66 \text{ (substituindo por 24)} \\ &= 3 \cdot 420 - 19 \cdot 66 \end{aligned}$$

o que revela a combinação linear de 420 e 66 que fornece o valor 6. ●

O algoritmo de Euclides nos fornece um modo de expressar mdc(a, b) como combinação linear de a e b, mas existe outro modo de caracterizar essa combinação linear.

TEOREMA **SOBRE MDC(a, b)**

Dados dois inteiros positivos a e b, mdc(a, b) é a combinação linear de a e b que tem o menor valor positivo.

Para provar esse resultado, precisamos usar o princípio da boa ordenação mencionado na Seção 2.2, a saber, que toda coleção de inteiros positivos que tem algum elemento tem um menor elemento. Pelo princípio da boa ordenação, essa coleção tem um menor elemento $c = ia + jb$, em que i e j são inteiros. O teorema afirma que c = mdc(a, b), ou seja, $c \mid a$, $c \mid b$ e c é o maior inteiro que divide tanto a quanto b.

Para provar que $c \mid a$, vamos fazer uma demonstração por absurdo. Suponha que $c \nmid a$. Então a divisão de a por c tem resto não nulo,

$$a = mc + r \quad \text{em que } m \text{ é inteiro e } 0 < r < c$$

Reescrevendo essa equação, temos

$$\begin{aligned} r &= a - mc \\ &= a - m(ia + jb) \\ &= (1 - mi)a - (mj)b \end{aligned}$$

o que mostra que r é uma combinação linear positiva de a e b, ou seja, r é um elemento da nossa coleção; mas $r < c$, o que é uma contradição, já que c é o menor elemento nessa coleção. Portanto, $c \mid a$. Podemos mostrar que $c \mid b$ da mesma forma. Então c é um divisor comum de a e de b; pelo Problema Prático 11, c é o máximo divisor comum de a e b, o que completa a demonstração do teorema.

PROBLEMA PRÁTICO 11

a. Prove que, se d for um inteiro positivo tal que $d \mid a$ e $d \mid b$, então $d \mid c$, em que $c = ia + jb$.
b. Prove que, se $d \mid c$, então $c \geq d$. ■

DEFINIÇÃO **PRIMOS ENTRE SI**

Dois inteiros a e b são **primos entre si** se mdc(a, b) = 1.

Do teorema sobre mdc(a, b), segue que a e b serão primos entre si se e somente se existirem inteiros i e j tais que

$$ia + jb = 1$$

PROBLEMA PRÁTICO 12 | Os inteiros 21 e 16 são primos entre si. Encontre i e j tais que $i(21) + j(16) = 1$. ■

Lembre-se de que um número primo é um inteiro $p > 1$ que só é divisível por 1 e p. Se a for um inteiro múltiplo de um primo p, é claro que $p \mid a$ e $p \mid p$, de modo que mdc(a, p) = p. Mas, se a não for um múltiplo de p, então mdc(a, p) = 1, já que nenhum outro número divide p. Portanto, dado qualquer inteiro, ou ele é um múltiplo de p ou ele e p são primos entre si.

Suponha que p é um número primo que divide o produto ab de dois inteiros a e b. Como p é "irredutível", p tem que dividir um dos números, a ou b. Mais formalmente, se p não dividir a, então a e p serão primos entre si, mdc(a, p) = 1 e existirão inteiros i e j tais que

$$1 = ia + jp$$

Multiplicando essa equação por b, obtemos

$$b = (ia)b + (jp)b = i(ab) + (jp)b$$

Como $p \mid ab$, podemos escrever ab na forma kp, em que k é um inteiro, de modo que a equação anterior fica

$$b = i(kp) + (jp)b = (ik + jb)p \qquad ik + jb \text{ inteiro}$$

e $p \mid b$. Isso prova o teorema a seguir.

TEOREMA **SOBRE A DIVISÃO POR NÚMEROS PRIMOS**

Seja p um número primo tal que $p \mid ab$, em que a e b são inteiros. Então $p \mid a$ ou $p \mid b$.

PROBLEMA PRÁTICO 13 | Estenda o teorema sobre a divisão por números primos da seguinte maneira: Seja p um número primo tal que $p \mid a_1a_2 \cdot \cdot \cdot a_k$, em que cada a_i é um inteiro. Então $p \mid a_j$ para algum j, $1 \le j \le k$. (*Sugestão*: Você quer provar que isso é verdade para todo inteiro positivo k. Qual técnica de demonstração deve usar?) ■

Finalmente, estamos prontos para demonstrar a unicidade (a menos de ordem) da fatoração de um número composto $n > 2$ como produto de números primos. Se n for um número composto, n poderá ser escrito como um produto de primos:

$$n = p_1p_2 \ldots p_r \quad \text{em que } p_1 \le p_2 \le \ldots \le p_r \text{ e cada } p_i \text{ é um número primo}$$

Agora suponha que n também pode ser escrito como

$$n = q_1q_2 \ldots q_s \quad \text{em que } q_1 \le q_2 \le \ldots \le q_s \text{ e cada } q_i \text{ é um número primo}$$

Então

$$p_1p_2 \ldots p_r = q_1q_2 \ldots q_s$$

Estamos supondo que essas duas representações são diferentes, mas elas podem ter alguns fatores em comum; vamos supor que os fatores em comum foram retirados pela divisão. Segue que

$$p_1 \mid p_1p_2 \ldots p_r$$

assim

$$p_1 \mid q_1q_2 \ldots q_s$$

Pelo Problema Prático 13, $p_1 \mid q_i$ para algum i, $1 \le i \le s$. No entanto, q_i é um número primo, divisível apenas por ele mesmo e por 1, portanto $p_1 = q_i$. Isso é uma contradição, já que tínhamos eliminado os fatores comuns.

EXEMPLO 31

Para encontrar a fatoração única de 825 como um produto de primos, podemos começar simplesmente dividindo 825 por números primos cada vez maiores (2, 3, 5 e assim por diante).

$$2 \nmid 825$$

$$825 = 3 \cdot 275 = 3 \cdot 5 \cdot 55 = 3 \cdot 5 \cdot 5 \cdot 11 = 3 \cdot 5^2 \cdot 11$$

Analogamente,

$$455 = 5 \cdot 7 \cdot 13$$

Dessas fatorações, podemos ver que mdc(825, 455) = 5. A decomposição de dois números inteiros nas suas respectivas fatorações em números primos é outra maneira (além do algoritmo de Euclides) de determinar seu máximo divisor comum.

PROBLEMA PRÁTICO 14 | Encontre a fatoração única de 1176 como um produto de primos. ■

PROBLEMA PRÁTICO 15 | Encontre mdc(420, 66) usando a fatoração única como produto de primos de cada um deles (veja o Exemplo 27). ■

Completamos a demonstração do teorema fundamental da aritmética. No entanto, esse é um resultado de *existência*. Ele diz que, para qualquer inteiro $n > 2$ que não é primo, existe uma única fatoração como produto de primos. Mas não nos diz

a. como decidir se n é primo;
b. se n não for primo, como encontrar os fatores primos de n.

Nenhum desses problemas tem uma solução algorítmica eficiente. A abordagem no Exemplo 31 de dividir um número n por primos cada vez maiores resolve os dois problemas — se n tiver fatores primos, eles serão descobertos, e se não existirem tais fatores, n será primo. No entanto, essa abordagem demandará um trabalho enorme quando n for grande.

Mais sobre Números Primos

Qualquer coisa com um título tão imponente como o teorema fundamental da aritmética tem que ser muito importante. Podemos usar esse teorema para descobrir muitos outros resultados sobre números primos.

Dado um inteiro positivo n, suponha que procuramos fatores primos dividindo n por números primos cada vez maiores. É claro que podemos parar quando chegamos ao maior primo menor ou igual a n, mas, de fato, podemos parar antes, quando chegar ao maior primo menor ou igual a $\sqrt{n}$. Se n puder ser fatorado de maneira não trivial como $n = st$, então tanto s quanto t não podem ser maiores do que $\sqrt{n}$, senão seu produto seria maior do que n. Logo, um deles, digamos s, tem que ser menor ou igual a $\sqrt{n}$. Pelo teorema fundamental da aritmética, s é um primo ou pode ser escrito como um produto de primos. Em qualquer dos casos, existe um fator primo menor ou igual a $\sqrt{n}$, o que prova o teorema a seguir.

TEOREMA SOBRE O TAMANHO DOS FATORES PRIMOS

Se n for um número composto, então terá um fator primo menor ou igual a $\sqrt{n}$.

EXEMPLO 32

Dado $n = 1021$, vamos encontrar os fatores primos de n ou determinar que n é primo. O valor de $\sqrt{1021}$ é um pouco menor do que 32. Então os primos que precisamos testar são 2, 3, 5, 7, 11, 13, 17, 19, 23, 29 e 31. Como nenhum deles divide 1021, 1021 é primo. ●

Quantos primos existem? Uma infinidade.

TEOREMA SOBRE A INFINIDADE DE PRIMOS (EUCLIDES)

Existe um número infinito de números primos.

Demonstração: Suponha que exista um número finito de números primos, listados como $p_1, p_2, ..., p_k$. Considere o número $s = p_1p_2 \ldots p_k + 1$. O inteiro s é maior do que todos os números primos $p_1, p_2, \ldots, p_k$, que estamos supondo são todos os primos, logo s não é primo. Então s é um número composto, e, pelo teorema fundamental da aritmética, s tem uma fatoração como um produto de (alguns dos) números primos. Suponha que p_j é um dos fatores primos de s, ou seja, $s = p_j(m)$ para algum inteiro m. Temos

$$1 = s - p_1p_2 \ldots p_k = p_j(m) - p_1 \ldots p_k = p_j(m - p_1 \ldots p_{j+1} \ldots p_{j+1} \ldots p_k)$$

Portanto $p_j \mid 1$, uma contradição. *Fim da demonstração*

Embora o teorema de Euclides diga que sempre há outro número primo à frente ao seguirmos ao longo dos números inteiros positivos, a distribuição de primos nos inteiros é errática. Ao contrário do que poderíamos imaginar, os primos não vão ficando cada vez mais distantes uns dos outros. Mesmo entre os primos pequenos, 23 e 29 distam mais do que 29 e 31.

EXEMPLO 33

No Exemplo 11, demonstramos por absurdo que $\sqrt{2}$ não é um número racional. O mesmo argumento funciona para $\sqrt{3}$ e $\sqrt{5}$ (2, 3 e 5 são todos números primos).

Podemos generalizar esse resultado de um único primo para qualquer inteiro x que seja um produto de um número ímpar de fatores primos. Suponha que $x = p_1p_2 \ldots p_{2k+1}$, pelo teorema fundamental da aritmética. (Neste exemplo, não estamos usando expoentes, de modo que alguns desses primos podem ser iguais; ou seja, 75 = 3*5*5.) Fazendo, mais uma vez, uma demonstração por absurdo, suponha que $\sqrt{p_1p_2 \cdots p_{2k+1}}$, em que a e b são inteiros, $b \neq 0$, a e b primos entre si. Então

$$p_1p_2 \cdots p_{2k+1} = \frac{a^2}{b^2}$$

$$a^2 = p_1p_2 \cdots p_{2k+1}b^2$$

Pelo teorema fundamental, a pode ser escrito como um produto de um ou mais primos, mas a_2 irá acrescentar outro fator para cada fator primo de a, resultando em um número par de fatores primos. Analogamente, b^2 é um produto de um número par de fatores

primos. Portanto, $p_1p_2 \ldots {}_{p2k+1b}{}^2$ é um produto de um número ímpar com um número par de fatores primos (o que nos dá um número ímpar de fatores primos). Mas então a_2 tem um número par de fatores primos, enquanto $p_1p_2 \ldots p_{2k+1}b^2$ tem um número ímpar de fatores primos, o que contradiz a unicidade do teorema fundamental. ●

A busca por números primos e por informações sobre números primos tem gerado muito interesse. Até junho de 2013, o maior número primo conhecido era $2^{57.885.161} - 1$, um número com 17.425.170 dígitos. Se considerarmos que, para uma fonte de tamanho razoável, uma impressora usa da ordem de 25 mm para imprimir 10 algarismos, seriam necessários mais de 43,5 km para imprimir um número desse tamanho!

Uma das conjecturas mais antigas sobre números primos — ainda sem solução — é a *conjectura de Goldbach*, formulada em 1742: Todo inteiro par maior do que 2 é a soma de dois números primos.

A Função Fi de Euler

DEFINIÇÃO **FUNÇÃO FI DE EULER**

Para um inteiro $n \geq 2$, a **função *fi* de Euler** de n, $\varphi(n)$, é o número de inteiros positivos k menores ou iguais a n tais que k e n são primos entre si. (φ é a letra grega "*fi*".)

Como n e n nunca são primos entre si, essa definição poderia ter sido enunciada como "o número de inteiros positivos k menores do que n tais que k e n são primos entre si", mas ocorre que é mais conveniente incluir a igualdade.

EXEMPLO 34

Os primeiros valores de $\varphi(n)$, junto com os números que fornecem esses valores, são

$\varphi(2) = 1$ (o número 1)
$\varphi(3) = 2$ (os números 1, 2)
$\varphi(4) = 2$ (os números 1, 3)
$\varphi(5) = 4$ (os números 1, 2, 3, 4)
$\varphi(6) = 2$ (os números1, 5)
$\varphi(7) = 6$ (os números 1, 2, 3, 4, 5, 6) ●

PROBLEMA PRÁTICO 16 Seja p um número primo. Prove que $\varphi(p) = p - 1$. ■

Para n pequeno, é fácil calcular $\varphi(n)$ usando uma abordagem de força bruta, tentando todos os valores $k < n$ para verificar quais têm a propriedade de que k e n são primos entre si. Mas, de fato, existe uma fórmula para calcular $\varphi(n)$, que deduziremos a seguir.

EXEMPLO 35

Usando o teorema fundamental da aritmética, escreva o inteiro positivo n como um produto de primos de modo que, se o mesmo primo p ocorrer m vezes, aparecerá como p^m. Suponha, por exemplo, que

$$n = p_1^{m_1} p_2^{m_2} p_3^{m_3}$$

Para calcular $\varphi(n)$, precisamos contar todos os inteiros positivos $k \leq n$, dos quais existem n, e eliminar todos os k tais que k e n não são primos entre si. Seja A_i, $1 \leq i \leq 3$, a coleção de todos os múltiplos inteiros positivos de p_i que são $\leq n$; esses números e n têm um fator comum, logo eles e n não são primos entre si. Os múltiplos inteiros de p_i que são $\leq n$ são p_i, $2p_i$, $3p_i$, . . . , n. Quantos números tem essa lista? Exatamente o número de vezes que você pode dividir n por p_i, ou seja, n/p_i. Então, denotando o número de elementos em A_i por $|A_i|$, sabemos que $|A_i| = n/p_i$. Se combinarmos A_1, A_2 e A_3, teremos todos os inteiros positivos $k \leq n$ tais que k e n são primos entre si. Não podemos simplesmente somar $|A_1| + |A_2| + |A_3| = n/p_3 + n/p_2 + n/p_1$, pois aqui existem números que pertencem a mais de um desses conjuntos e seriam contados mais de uma vez. Os números que aparecem tanto em A_i quanto em A_j com $i \neq j$ são os que são múltiplos inteiros de p_i e de p_j, logo teremos $n/(p_i\, p_j)$ deles. Temos que subtrair esses para todas as combinações i, j. Mas, ao fazer isso, subtraímos todos os números que pertencem aos três conjuntos três vezes, de modo que eles não estão mais sendo contados, que assim temos que somar os $n/(p_1\, p_2\, p_3)$ números que pertencem aos três conjuntos.[2] Portanto,

$$\varphi(n) = n - \left(\frac{n}{p_1} + \frac{n}{p_2} + \frac{n}{p_3} - \frac{n}{p_1p_2} - \frac{n}{p_2p_3} - \frac{n}{p_1p_3} + \frac{n}{p_1p_2p_3}\right)$$

$$= n\left(1 - \frac{1}{p_1} - \frac{1}{p_2} - \frac{1}{p_3} + \frac{1}{p_1p_2} + \frac{1}{p_2p_3} + \frac{1}{p_1p_3} - \frac{1}{p_1p_2p_3}\right)$$

$$= n\left(\frac{p_1p_2p_3 - p_2p_3 - p_1p_3 - p_1p_2 + p_3 + p_1 + p_2 - 1}{p_1p_2p_3}\right) \quad \text{(somando as frações)}$$

$$= n\left(\frac{(p_1 - 1)(p_2 - 1)(p_3 - 1)}{p_1p_2p_3}\right) \quad \text{(verifique isso expandindo o numerador)}$$

$$= \frac{n}{p_1p_2p_3}(p_1 - 1)(p_2 - 1)(p_3 - 1)$$

$$= p_1^{m_1-1}p_2^{m_2-1}p_3^{m_3-1}\varphi(p_1)\varphi(p_2)\varphi(p_3) \qquad (1)$$

A Equação (1) expressa $\varphi(n)$ em termos dos valores da função fi de Euler de seus fatores primos, que são conhecidos (veja o Problema Prático 16).

A Equação (1) no Exemplo 34 fornece uma fórmula para $\varphi(n)$ quando n tem três fatores primos distintos. É fácil estender essa equação ao caso mais geral quando n tem um número arbitrário de fatores primos. Se

$$n = p_1^{m_1}p_2^{m_2}\cdots p_k^{m_k}$$

então

$$\varphi(n) = p_1^{m_1-1}p_2^{m_2-1}\cdots p_k^{m_k-1}[\varphi(p_1)\varphi(p_2)\cdots\varphi(p_k)] \qquad (2)$$

EXEMPLO 36

Para $n = 133.848 = 2^3 \cdot 3^2 \cdot 11.\ 13^2$,

$\varphi(n) = 2^2 \cdot 3 \cdot 13[\varphi(2)\varphi(3)\varphi(13)] = 4 \cdot 3 \cdot 13[1 \cdot 2 \cdot 10 \cdot 12] = 37440$

Para usar a Equação (2), é preciso conhecer a fatoração de n em números primos, logo isso não evita a dificuldade que observamos anteriormente sobre a fatoração de números grandes.

PROBLEMA PRÁTICO 17 | Para $n = 3^4 \cdot 5 \cdot 72$, calcule $\varphi(n)$. ■

SEÇÃO 2.4 REVISÃO

TÉCNICAS

- Representação do mdc(a, b) como uma combinação linear de a e b.
- Verificação se um número é ou não primo e, caso não seja, cálculo de sua fatoração em primos.
- Cálculo da função fi de Euler $\varphi(n)$ para um inteiro positivo n.

IDEIAS PRINCIPAIS

- Todo inteiro $n \geq 2$ é primo ou tem uma fatoração única como produto de primos (teorema fundamental da aritmética).
- Dois inteiros positivos a e b serão primos entre si se existir uma combinação linear de a e b igual a 1.
- Se n não for primo, terá um fator primo menor ou igual a $\sqrt{n}$.
- Existe um número infinito de números primos.
- Não existe um algoritmo eficiente para decidir se um número é primo ou para encontrar uma fatoração em primos, caso não seja.
- Para n dado como um produto de primos, existe uma fórmula para calcular $\varphi(n)$, o número de inteiros positivos $k \leq n$ tais que k e n são primos entre si.

EXERCÍCIOS 2.4

Os Exercícios 1 a 6 se referem aos Exercícios de 7 a 12 na Seção 2.3.

1. Escreva mdc(308, 165) como uma combinação linear de 308 e 165.
2. Escreva mdc(2420, 70) como uma combinação linear de 2420 e 70.
3. Escreva mdc(735, 90) como uma combinação linear de 735 e 90.
4. Escreva mdc(8370, 465) como uma combinação linear de 8370 e 465.
5. Escreva mdc(1326, 252) como uma combinação linear de 1326 e 252.
6. Escreva mdc(1018215, 2695) como uma combinação linear de 1018215 e 2695.